QueryAPIGate DOCUMENTATION · PERFORMANCE

Performance

Current as of QueryAPIGate **v0.49.0** · PDF made 4 November 2026

Latest version: anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?d=PERFORMANCE

> **For 0.43's figures, read Performance, v4 (PostgreSQL) and Performance on MySQL, v2.** For the method, Performance, v3. For the Rust data plane in front of `/q/` (ADR 0028), read Performance, v5.
> **For PostgreSQL, read Performance, v3 instead.** It's a more careful run: pinned cores, 5 interleaved rounds per comparison with their ranges, open-loop load, a simulated network, a measured round-trip count, and a profile. It also corrects two figures on this page. A call's statements on the store went from 7 to 3, not "about 5.5 to about 2.6". And the PostgreSQL throughput here is about half what that machine gives in a steady state.

The short answer: QueryAPIGate adds a millisecond or two to a request, and almost none of that is the governance. In front of PostgreSQL, one container in its default configuration serves about 300–400 requests a second, reads or writes, with a key's masking and table allow-list on or off. Throughput grows with the number of containers. The database is rarely the limit: in these tests PostgreSQL used about a core or less per container.

This page is what we measured, where the time goes, and how to measure it on your own hardware. Every number comes from one laptop, so read them as a guide to proportions, not a promise. The same laptop measured about half these numbers earlier on the same day, even on `/health`, which does no work. Compare numbers within a table, not across tables or with your own machine.

How it was measured

- **The machine:** a 20-thread Intel i7-12700H laptop with 32 GB of memory. The load generator, QueryAPIGate and the database all ran on it, so there's no network hop, and the load generator takes CPU from the server.
- **The server:** QueryAPIGate as the Docker image runs it: gunicorn, **one worker with eight threads** (`--worker-class gthread`), the access log on. For Scaling out, 2, 4 and 8 workers.
- **The load:** oha, a 3-second warm-up, then 10 seconds per run. For PostgreSQL, each request asks for a random row, so nothing is answered from a cache the test made warm.
- **Two setups:**
  - the bundled example APIs on SQLite, to show the gateway's own cost on a database that answers in microseconds;
  - an OLTP database on PostgreSQL 16 - pgbench's 10-million-row bank schema, with QueryAPIGate's own store in the same PostgreSQL, as a production deployment would have it.

Measured 2026-10-23 on `main` after 0.41.0, with the changes this page describes.

The example APIs on SQLite

- **Two queries:**
  - a **primary-key lookup** (`SELECT film_id, title FROM film WHERE film_id = :film_id`), so the gateway's own cost shows;

  - `example_film_lookup`, a **join and aggregate** over the rentals table, as a realistic query.
- **Two kinds of key:**
  - **plain** - a scoped key with no masking and no table allow-list;
  - **governed** - the same key with `allowed_tables` and a masking rule that hashes `title` on every row.

Requests a second, and latency in milliseconds:

CALL	CONCURRENCY	REQ/S	P50	P99
`/health` - the floor	1	1,583	0.6	1.3
Primary-key lookup, plain key	1	839	1.2	1.9
Primary-key lookup, **governed key**	1	778	1.2	2.4
Join query, plain key	1	526	1.9	2.9
Join query, **governed key**	1	510	1.9	3.1
Join query, governed key	8	583	12.6	26.7
Join query, governed key	32	505	61.8	92.5

A governed call costs about the same as a plain one. Masking and the table check are worked out once per statement and reused, so on a repeated call they add almost nothing.

Throughput stays flat as concurrency rises; latency is what grows. One worker process runs Python code on one core at a time, so extra concurrent callers wait their turn: 32 callers sharing about 500 requests a second each wait about 60 ms. That's why you scale by adding processes rather than threads (see Scaling).

An OLTP database on PostgreSQL

The database

pgbench's TPC-B-like schema at scale 100 (`pgbench -i -s 100`): 1.5 GB, in a `postgres:16` container started with `-c shared_buffers=2GB -c max_connections=300` - the whole database fits in memory, so this measures the gateway, not the disk.

TABLE	ROWS	COLUMNS	KEY
`pgbench_accounts`	10,000,000	`aid integer`, `bid integer`, `abalance integer`, `filler character(84)`	primary key `aid`
`pgbench_branches`	100	`bid integer`, `bbalance integer`, `filler character(88)`	primary key `bid`
`pgbench_tellers`	1,000	`tid integer`, `bid integer`, `tbalance integer`, `filler character(84)`	primary key `tid`
`pgbench_history`	0 at the start	`tid`, `bid`, `aid`, `delta integer`, `mtime timestamp`, `filler character(22)`	none

QueryAPIGate's own store - keys, saved queries, connections, run history - was a second database, `qagstore`, in the same PostgreSQL (`QUERYAPIGATE_DATABASE_URL`).

The calls

Three saved queries, run through `/q/<name>` with a random id on every request:

SAVED QUERY	SQL	CALLED AS
`account` - a **lookup**	`SELECT aid, bid, abalance FROM pgbench_accounts WHERE aid = :aid`	`/q/account?aid=` 1 to 9,999,999
`account_page` - a **100-row page** with a join	`SELECT a.aid, a.abalance, b.bid, b.bbalance FROM pgbench_accounts a JOIN pgbench_branches b ON b.bid = a.bid WHERE a.aid BETWEEN :start AND :start + 99 ORDER BY a.aid`	`/q/account_page?start=` 1 to 999,999 `&page_size=100`
`deposit` - a **write**	`UPDATE pgbench_accounts SET abalance = abalance + :delta WHERE aid = :aid`	`/q/deposit?aid=` 1 to 9,999,999 `&delta=` 1 to 9

And three keys:

- **plain** - granted `account` and `account_page`, nothing else;
- **governed** - the same, plus `allowed_tables` (`pgbench_accounts`, `pgbench_branches`) and a masking rule that hashes `*.abalance`;
- **writer** - granted `deposit`, with `allow_writes` and `allowed_write_ops: ["update"]`, on a server started with `QUERYAPIGATE_ALLOW_WRITES=1`.

Every request in every run below returned 200, and the writes landed.

The database on its own

pgbench against the same container, for scale:

WORKLOAD	CLIENTS	TRANSACTIONS A SECOND	AVERAGE LATENCY
Primary-key read (`pgbench -S`)	1	11,821	0.085 ms
Primary-key read (`pgbench -S`)	32	221,481	0.144 ms
TPC-B transaction - three updates, a select and an insert	32	20,869	1.5 ms

A lookup costs PostgreSQL under a tenth of a millisecond. Whatever a call through the gateway takes beyond that is the gateway.

Through the gateway: one container

One worker, eight threads, 32 concurrent callers; requests a second. *Before* is the request path before the changes to the store reads and the pool below. *Now* is this version's, at its defaults. *Now, with the metadata cache* adds `QUERYAPIGATE_METADATA_CACHE_TTL=5` (see The metadata cache).

CALL	BEFORE	NOW	NOW, WITH THE METADATA CACHE
Lookup, plain key	326	368	430
Lookup, governed key	343	338	392
100-row page	306	318	346
Write	326	368	420

The lookup row is the median of three rounds, run interleaved; the others are one run each. Runs varied by up to about 10% between rounds, so a smaller difference than that, such as the governed lookup's, isn't one.

Latency for one caller at a time, the lookup with a plain key, median of three rounds:

	REQ/S	P50
Before	414	1.9 ms
Now	440	1.8 ms
Now, with the metadata cache	692	1.2 ms

Scaling out in front of PostgreSQL

Requests a second with 64 concurrent callers, by the number of gunicorn workers - each one a process like a container's:

WORKERS	BEFORE: LOOKUPS / WRITES	NOW: LOOKUPS / WRITES	NOW, WITH THE METADATA CACHE: LOOKUPS / WRITES
1	339 / 380	419 / 410	473 / 464
2	1,094 / 921	933 / 928	1,144 / 1,157
4	1,336 / 1,308	1,444 / 1,447	1,876 / 1,830
8	1,926 / 1,763	2,093 / 2,078	2,732 / 2,650

- **Processes add throughput.** Eight of them served five to six times what one did, on the same database.
- **One process with 64 callers does worse than its share.** The callers queue for its eight threads, which is why the step from one worker to two is more than double. Don't size capacity from one overloaded process.
- **The metadata cache matters more as you scale out.** With 8 workers, it's 30% more requests a second, and the store's PostgreSQL has far less to do.
- **These are workers on one 20-thread laptop**, sharing it with the database and the load generator. Containers on their own hosts would scale more cleanly.

Where the time goes

On SQLite, each part of a call, measured on its own:

PART OF A CALL	PRIMARY-KEY LOOKUP	JOIN QUERY
The database query itself	0.009 ms	0.26 ms
The read-only SQL check	0.01 ms	0.03 ms
The table check (`allowed_tables`), first call for a statement	0.3 ms	1.5 ms
Masking's trace of the result columns, first call for a statement	6.9 ms	28.9 ms

The first call for a statement pays for the table check and the masking trace; after that, both are reused. The table check depends only on the query text, so it can't go stale. The masking trace depends on the connection's schema too, so it is kept only as long as QueryAPIGate keeps that schema - at most 60 seconds - and straight away when the connection is changed. From 0.43 a busy connection's schema is read again in the background after 45 seconds, while calls go on using the cached copy; when it hasn't changed, the traces are kept. So no call waits for the read or the trace again - before, one call a minute took about 75 ms longer (Performance, v3). A connection with no masked call for a whole minute is read again in its next call, as before.

The rest is the gateway itself, the same for every query: authentication, limits, the record of the run, the HTTP response. Against a database that takes 50 ms to answer, it hardly matters; against one that answers in 0.01 ms, it's nearly the whole call. Serialization is small at typical page sizes - about 0.1 ms here.

Part of that fixed cost is QueryAPIGate's own store. Every call reads the key its secret belongs to, the saved query and the connection, and adds a run to the history. Transactions on the store, per call, counted in PostgreSQL's `pg_stat_database` over 200 lookups:

	STORE TRANSACTIONS PER CALL
Before	about 5.5 - the saved query was read three times
Now	about 2.6 - one read each for the key, the query and the connection; history in batches
Now, with the metadata cache	about 0 - only the batched history writes, about one a second

The data database saw about one transaction per call throughout: the query itself.

What governance costs

Column masking has to know which source column each result column comes from, so it traces the query through its aliases, joins, subqueries and unions - a few milliseconds of SQL analysis. The table check parses the query to find the tables it reads. Before the caches, both ran on every call, and a governed key was 4-8 times slower than a plain one.

Caching doesn't change what is masked or what is refused. A query that can't be traced or checked is refused every time, the same as before. Both caches have a fixed size, so a key sending many different ad-hoc queries can't make them grow without bound.

The metadata cache

`QUERYAPIGATE_METADATA_CACHE_TTL` (seconds, off by default) keeps the rows a call reads from the store - the API key, the saved query, the connection - in memory for that long, instead of reading them again on every call. In the tests above, it took the store out of a call's path: 10-20% more throughput for a container, and a third off the latency of a single call (1.8 ms to 1.2 ms).

What it caches is the rows, never a decision: whether a key is active, expired, used from an allowed address or still inside a rotation's grace period is checked on every call, against the cached row. What it can delay is a change:

- **A change made through the same server** - the Console, the Management API - is seen at once: every change to the store clears this server's cache.
- **A change made anywhere else** - another instance, another gunicorn worker, the `queryapigate` CLI, an edit straight to the database - is seen up to one TTL later. A key revoked on one instance keeps working on the others for that long.

So it is a trade: a few seconds of delay for a revocation that reaches other instances, for fewer reads on the store. Leave it off if a revoked key must stop working everywhere at once; with one instance, a TTL of a few seconds changes nothing anyone would notice.

Where it is set:

SETUP	`QUERYAPIGATE_METADATA_CACHE_TTL`	WHY
The server and the Docker image	`0`, off	The image runs anywhere, often several instances at a time
The one-command Compose demo (`docker-compose.yml`)	`5`	One instance: only a change made with the CLI waits, up to 5 s
The scale-out Compose setup (`deploy/scale-out/`)	off	Three instances: a revocation would take up to a TTL to reach the other two
The Helm chart	off; `config.QUERYAPIGATE_METADATA_CACHE_TTL`	Set it with one replica, or when the delay is acceptable

Change it for any setup with the variable; the Console's Settings screen shows the value in use.

Scaling

- **More containers, not more threads.** Each container is one Python process. Run several behind a load balancer on a shared PostgreSQL store and Redis, and throughput grows with the number of containers - see Scaling out and Run several instances behind a load balancer.
- **Why one worker per container.** `/metrics` is counted per process (and, without Redis, so are rate limits), so with several workers in one container a scraper would read one worker's numbers at random. One worker per container keeps every count complete.
- **The eight threads are for waiting.** They let one container keep serving while some calls wait on a slow database or a long-lived `/events` connection. They don't add CPU: in an earlier, slower run on the same laptop, one worker served 119, 169 and 164 requests a second with 1, 2 and 8 threads.
- **The database can take it.** In that earlier run, at 8 processes and 1,700 requests a second, PostgreSQL used about four cores - mostly for QueryAPIGate's own store, not the queries. The metadata cache takes most of that away.

Making it faster

- **Cache what you can.** A saved query with a `cache_ttl` is answered without the database for repeat calls.
- **Turn on the metadata cache** if a few seconds' delay for a change made on another instance is acceptable.
- **Tune the connection pool** for the number of callers - Diagnose and tune the connection pool. The default of 8 idle connections matches the Docker image's 8 threads. Below that, a busy server closes connections only to open new ones for the next calls.
- **Stream large results** with `?stream=true` instead of paging through them - see Streaming exports.
- **Look at the database first.** On anything bigger than a primary-key lookup, the query plan and indexes usually matter more than anything in the gateway.

What wasn't measured

MySQL and ClickHouse as the data source; a database across a network, with the round trip that comes with it; large result sets; the MCP server; and several containers behind a load balancer - the scaling numbers are several workers on one machine. Measure on your own hardware before you plan capacity from these numbers.

Measure it yourself

On the example APIs

Start a server the way the Docker image does, on the example APIs:

```shell
pip install "queryapigate[server,flow]"   # flow: the SQL analysis masking uses
mkdir -p /tmp/qag-bench
QUERYAPIGATE_HOME=/tmp/qag-bench QUERYAPIGATE_API_KEY=bench-admin QUERYAPIGATE_LOAD_EXAMPLES=1 \
  gunicorn --bind 127.0.0.1:5090 --workers 1 --worker-class gthread --threads 8 \
  --access-logfile /tmp/qag-bench/access.log -c python:queryapigate.gunicorn_conf
'queryapigate.app:create_app()'
```

Create a governed key, then put load on a query with it:

```shell
curl -s -X POST http://127.0.0.1:5090/api/v1/api-keys -H 'X-API-Key: bench-admin' -H 'Content-Type:
application/json' \
  -d '{"name": "bench-governed", "queries": ["example_film_lookup"], "connections": [],
       "allowed_tables": ["film", "rental"], "mask": [{"column": "*.title", "action": "hash"}]}'
# copy the "secret" from the response

docker run --rm --network host ghcr.io/hatoo/oha -z 10s -c 8 -H 'X-API-Key: <the secret>' \
  'http://127.0.0.1:5090/q/example_film_lookup?film_id=7'
```

Run a few seconds first as a warm-up - the first calls for each statement pay for the table check and the masking trace - then measure for at least 10 seconds: once with `-c 1` for latency on an idle server, then at higher concurrency for throughput. Point it at your own queries and database for numbers you can plan from.

On PostgreSQL

A PostgreSQL with pgbench's schema, and a second database for QueryAPIGate's store:

```shell
docker run -d --name qag-bench-pg -e POSTGRES_PASSWORD=pw -p 55433:5432 postgres:16 \
  -c shared_buffers=2GB -c max_connections=300
docker exec qag-bench-pg psql -U postgres -c 'CREATE DATABASE bench' -c 'CREATE DATABASE qagstore'
docker exec qag-bench-pg pgbench -U postgres -i -s 100 bench     # 10 million accounts, about 1.5 GB
docker exec qag-bench-pg pgbench -U postgres -S -c 32 -j 8 -T 10 bench   # the database on its own
```

QueryAPIGate on that store - `psycopg2` is in the `postgres` extra:

```shell
pip install "queryapigate[server,postgres]"
mkdir -p /tmp/qag-oltp
QUERYAPIGATE_HOME=/tmp/qag-oltp QUERYAPIGATE_API_KEY=bench-admin \
  QUERYAPIGATE_DATABASE_URL=postgresql://postgres:pw@127.0.0.1:55433/qagstore \
  gunicorn --bind 127.0.0.1:5091 --workers 1 --worker-class gthread --threads 8 \
  -c python:queryapigate.gunicorn_conf 'queryapigate.app:create_app()'
```

A connection, the lookup as a saved query, and a key for it:

```shell
A='X-API-Key: bench-admin'; J='Content-Type: application/json'; U=http://127.0.0.1:5091
curl -s -X POST $U/api/v1/connections -H "$A" -H "$J" -d '{"name": "oltp", "db": "postgres", "host":
"127.0.0.1",
  "port": 55433, "database": "bench", "user": "postgres", "password": "pw", "active": true}'
curl -s -X POST $U/api/v1/queries -H "$A" -H "$J" -d '{"name": "account", "connection_name": "oltp",
  "sql": "SELECT aid, bid, abalance FROM pgbench_accounts WHERE aid = :aid",
  "parameters": {"aid": {"type": "int", "required": true}}, "description": "One account", "publish":
true}'
curl -s -X POST $U/api/v1/api-keys -H "$A" -H "$J" -d '{"name": "bench", "queries": ["account"],
"connections": []}'
# copy the "secret" from the response
```

Then the load, with a random account on every request:

```shell
docker run --rm --network host ghcr.io/hatoo/oha -z 10s -c 32 -H 'X-API-Key: <the secret>' \
  --rand-regex-url 'http://127.0.0.1:5091/q/account\?aid=[1-9][0-9]{0,6}'
```

Restart gunicorn with `QUERYAPIGATE_METADATA_CACHE_TTL=5` to see the metadata cache, or with `--workers 4` to see a second process's worth. When you're done, `docker rm -fv qag-bench-pg` removes the container and its data.

READ NEXT

Threat model

QUESTIONS OR PROBLEMS

Discord · GitHub issues

LATEST VERSION OF THIS PAGE

anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?d=PERFORMANCE

FOLLOW QUERYAPIGATE

GitHub X Discord Bluesky Mastodon YouTube Reddit

@queryapigate on X, Bluesky, Mastodon and YouTube · r/queryapigate on Reddit · github.com/AnanthaRajuC/QueryAPIGate

QueryAPIGate · one governed gate between your databases and everything that asks them for data · self-hosted, Apache-2.0 · © 2026 QueryAPIGate